

# Unified Implementation Plan for Autonomous CNC Toolpath System

M365 Copilot

June 19, 2026

## 1 Included Source Prompts

### 1.1 Toolpath Prompt

*You are a software+mechanical engineer tasked with generating a toolpath generation software... tool must enter from outside the face.*

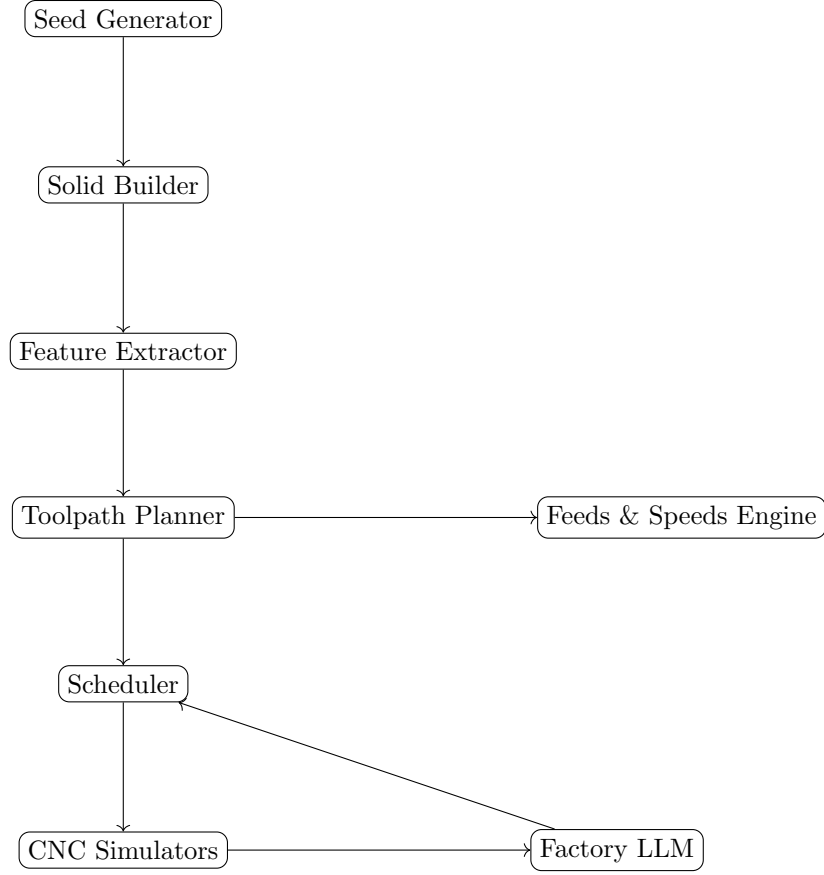
### 1.2 Feeds and Speeds Context

*You are selecting feeds and speeds based on material, tool, and operation with shop-floor adjustments.*

### 1.3 Factory Simulation Prompt

*You are developing a simulation of an autonomous factory with multiple CNC machines, toolpaths, scheduling, and optimization policies.*

## 2 System Architecture



## 3 Implementation Modules

### 3.1 1. Solid Builder

- Generate 1–4 cuboids (axis-aligned)
- Add pockets and holes
- Output: face list with wall/cliff metadata

### 3.2 2. Feature Extractor

- Identify top faces  $P_i$
- Label edges: Wall / Cliff / Level

### 3.3 3. Toolpath Planner

Core algorithm:

- Classify edges (wall, cliff) [?]
- Compute safe region  $S_i = P_i \ominus B_{blocked}$  [?]
- Generate raster paths
- Ensure entry from safe edge

### 3.4 4. Feeds & Speeds Engine

- Compute RPM from cutting speed [?]
- Compute feed:  $F = NZf_z$  [?]
- Adjust based on chatter, rigidity

### 3.5 5. Drilling & Entry Module

- Peck drilling rules (0.25D–1D) [?]
- Entry selection (plunge, ramp, helix)

### 3.6 6. Scheduler

- Assign parts to machines
- Optimize based on policy:
  - Min time
  - Max tool life
  - Min cost
  - Best finish

### 3.7 7. CNC Simulator

- Execute G-code
- Track:
  - Tool wear
  - Time
  - Errors

## 4 Algorithm Pipeline

seed -> build geometry -> extract features  
-> generate toolpath -> assign feeds/speeds  
-> schedule -> simulate -> feedback

## 5 Key Data Structures

- Face: polygon + height + edges
- Tool: diameter, life, cost
- Machine: queue, tool crib
- Feature: pocket, hole, face

## 6 Weaknesses and Missing Information

### 6.1 Geometry Limitations

- Only cuboids supported
- No curved surfaces or rotations

## 6.2 Toolpath Gaps

- No cutter deflection modeling
- No corner smoothing/blending

## 6.3 Physics Gaps

- No cutting force model
- No power/torque constraints

## 6.4 Feeds & Speeds Limitations

- Empirical only (no adaptive control)
- Missing tool coating effects

## 6.5 Simulation Gaps

- Tool crib not fully implemented
- Error handling incomplete
- Multi-machine coordination unclear

## 6.6 Optimization Gaps

- Multi-objective optimization not formalized
- No global optimization across features

## 6.7 Software Gaps

- Dependency on outdated OCC environment
- No API contract between modules

# 7 Recommended Improvements

- Add force and power models
- Implement adaptive feed control
- Extend geometry kernel
- Formalize API between modules
- Add unit tests and benchmarking

# 8 Hierarchical Error-Aware Multi-Agent Control System

## 8.1 Overview

To address error handling and multi-machine coordination, we introduce a hierarchical control architecture enabling real-time anomaly interpretation, safe intervention, and coordinated mitigation across CNC agents and the factory system.

## 8.2 Integrated Execution Loop

The base loop:

simulate -> feedback

is extended to:

simulate -> prompt -> transform -> CNC reasoning -> safety check -> escalate -> factory  
reasoning -> mitigate -> resume

## 8.3 8.2 Integrated Execution Loop (Component Definitions)

The execution loop is:

simulate → prompt → transform → CNC reasoning → safety check → escalate →  
factory reasoning → mitigate → resume

1. **Simulate** Advance machine and factory state by one control interval. This stage executes scheduled operations, updates tool wear, progresses jobs, and captures current telemetry (tick, machine status, queue depth, active tool, and error state). *Output:* Updated runtime state and raw operational signals.
2. **Prompt** Generate an event prompt when anomalous or decision-relevant conditions are detected (for example chatter, tool breakage risk, queue imbalance, or policy conflict). Prompts may originate from sensors, rules, or operator input. *Output:* Human-readable or structured trigger message.
3. **Transform** Convert prompt text and runtime context into a structured decision payload. This transformation standardizes fields such as event type, severity, affected machine, tool context, job context, and recent history. *Output:* Normalized context object for deterministic and LLM reasoning.
4. **CNC Reasoning** Perform local machine-level diagnosis and generate candidate actions (for example reduce feed, pause, replace tool, continue with caution). This stage prioritizes local constraints and machine safety boundaries. *Output:* Ranked local action candidates with rationale and confidence.
5. **Safety Check** Validate each candidate against hard safety rules and operational invariants. Unsafe candidates are rejected immediately. Safety-critical events may force immediate stop behavior. *Output:* Safety-filtered candidate set and risk labels (safe, risky, critical).
6. **Escalate** If local resolution is insufficient, high-risk, or has system-wide implications, escalate the case to factory-level coordination with full context package and local reasoning trace. *Output:* Escalation record linking machine context to factory-level decision flow.
7. **Factory Reasoning** Evaluate global tradeoffs across machines, queues, inventory, and policy objectives. Compute coordinated actions such as rerouting, policy tuning, maintenance sequencing, or controlled hot-swap proposals. *Output:* Factory-level mitigation plan with expected impact and rollback strategy.
8. **Mitigate** Execute approved corrective actions. For any hot-swap action, enforce mandatory human approval before execution. Log approval decision, action dispatch, and pre/post state snapshots for evaluation. *Output:* Applied mitigation commands, audit events, and verification checkpoints.
9. **Resume** Return machine and factory workflows to normal progression from a verified safe state. Monitor short-horizon outcomes to confirm resolution and trigger rollback if regression is detected. *Output:* Stable continuation of production with outcome metrics recorded.

### Operational Notes

- Every transition emits telemetry events for replay and post-incident analysis.
- Decision and outcome events are correlated for effectiveness scoring.
- Human approval is a hard gate for all hot-swap operations before mitigation execution.

## 8.4 Design Principles

- Separation of concerns: CNC-local vs Factory-global
- Structured context over raw prompt text
- Safety-first execution boundaries
- Human-in-the-loop supervision

# 9 Expanded Query, Telemetry, and Human-Approved Hot-Swap Architecture

## 9.1 Problem Statement

Current updates are concentrated in routing/policy/tool-optimization paths. The system needs:

- richer query support backed by runtime data,
- persistent telemetry for decision context,
- full action/event logs for post-event evaluation,
- strict governance where every hot-swap requires explicit human approval.

## 9.2 Mandatory Human Approval Rule

**Rule:** No hot-swap may execute without explicit human approval.

This applies to:

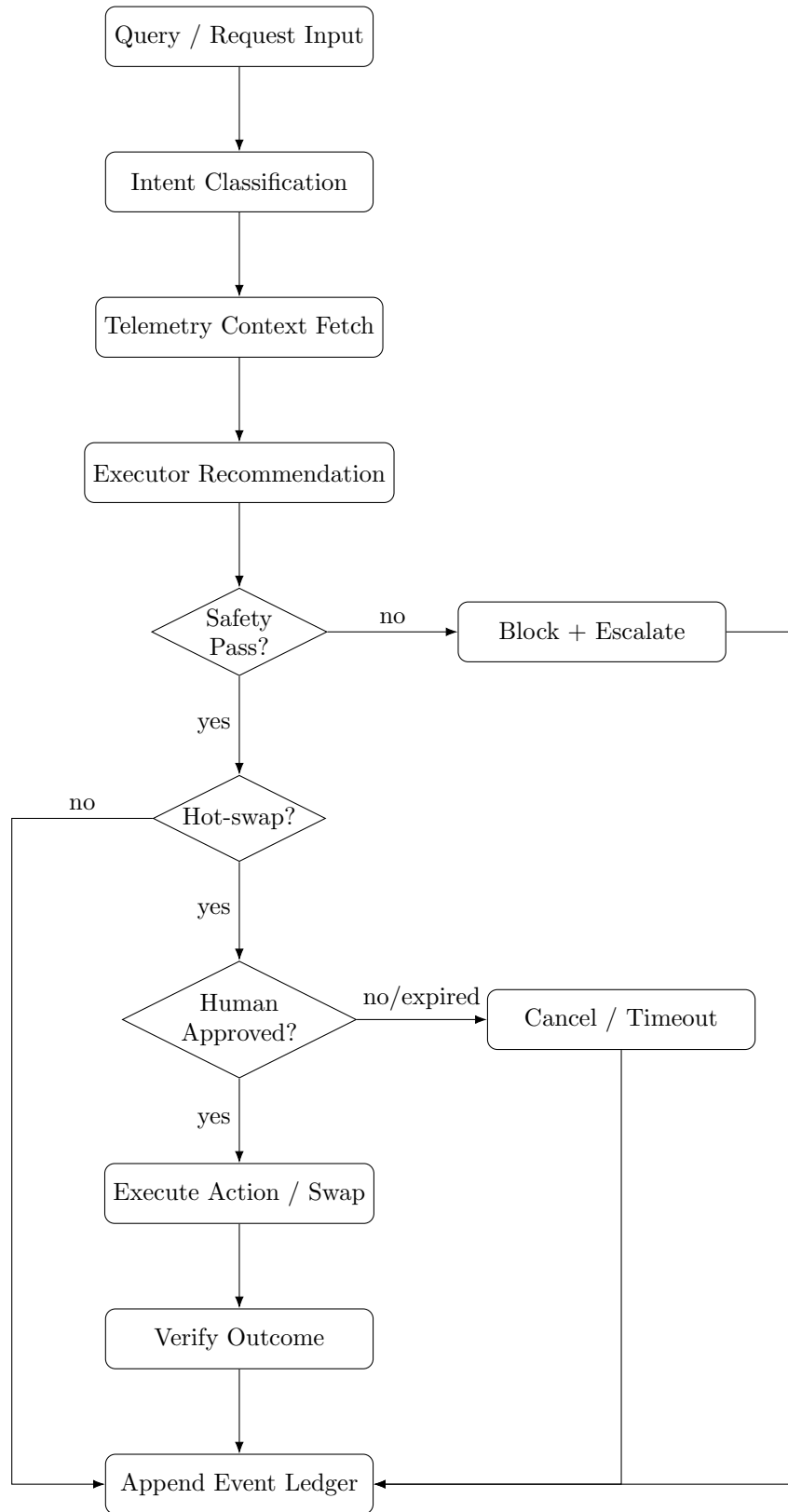
- runtime strategy swaps (routing, policy, tool parameters),
- plugin/module swaps (install, enable, disable, rollback).

Emergency stop/abort remains allowed without pre-approval because it is a safety control, not a hot-swap, but it must still be logged.

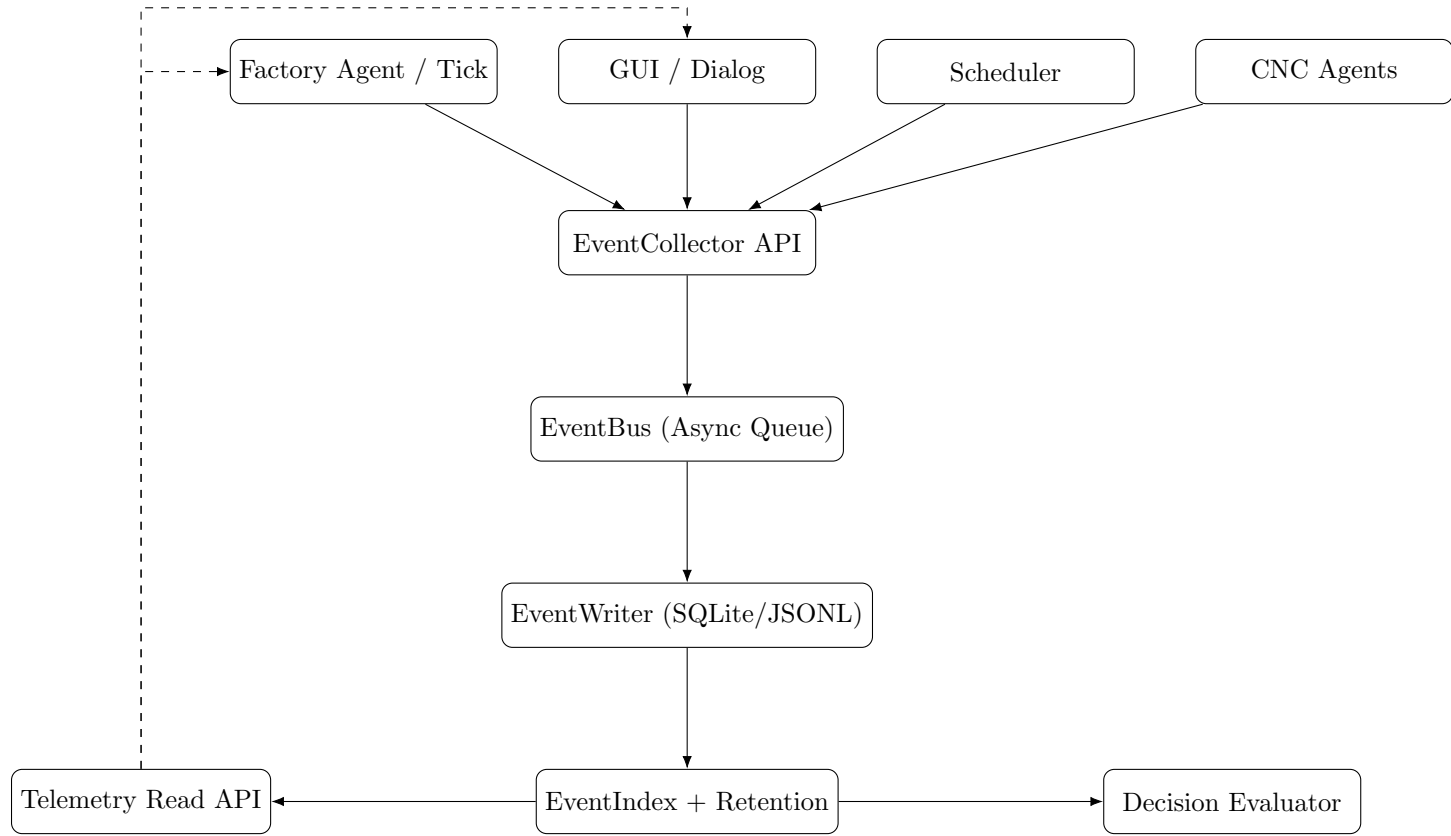
## 9.3 Expanded Query Taxonomy

- Observability queries: machine state, queue, tool crib, KPI.
- Deterministic control actions: scheduling/policy/routing controls.
- Advisory optimization actions: LLM + rule-based recommendations.
- Safety/emergency actions: stop, abort, escalation.
- Telemetry analytics queries: action effectiveness, trend analysis, incident replay.
- Plugin lifecycle actions: install/enable/disable/rollback (approval-gated).

## 9.4 Control Flow with Human Approval Gate



## 9.5 Data Collection and Query Supplementation Architecture



## 9.6 Unified Event Envelope

Every event uses:

| Field                             | Meaning                                |
|-----------------------------------|----------------------------------------|
| event_id, event_type              | Unique id and canonical event category |
| tick, timestamp_utc, wall_time_ms | Logical and wall-clock timing          |
| machine_id, job_id, seed          | Operational context                    |
| source, category, severity        | Producer identity and severity class   |
| correlation_id, causation_id      | Cross-event lineage for replay         |
| payload                           | Typed event-specific data              |

## 9.7 Mandatory Hot-Swap and Approval Events

- swap\_requested
- swap\_risk\_assessed
- swap\_approval\_requested
- swap\_approved / swap\_rejected / approval\_expired
- swap\_executed
- swap\_verified

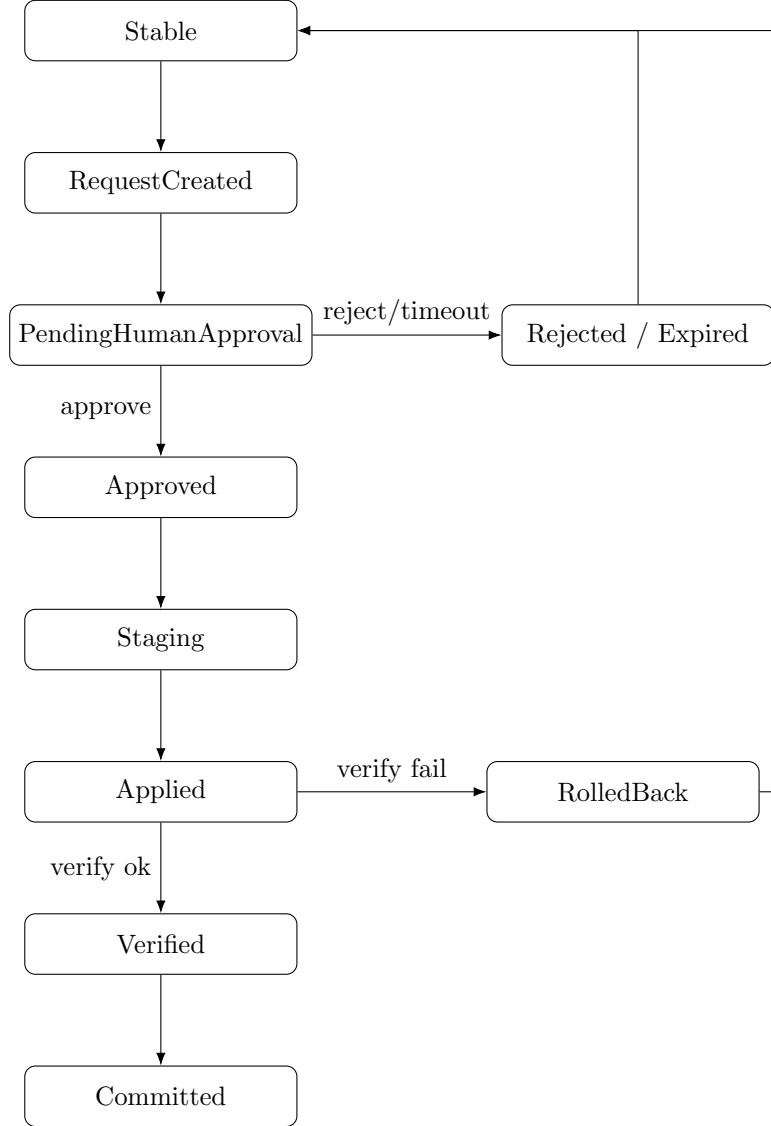


- swap\_rolled\_back
- override\_denied\_without\_approval

## 9.8 Human Approval Invariants

$$\begin{aligned}
& \text{ExecuteHotSwap}(r) \Rightarrow \text{ValidApprovalToken}(r) \\
& \text{ValidApprovalToken}(r) \Rightarrow (\text{scope\_match} \wedge \text{not\_expired} \wedge \text{approver\_present}) \\
& \neg \text{ValidApprovalToken}(r) \Rightarrow \text{SwapBlocked}(r)
\end{aligned}$$

## 9.9 Dual-Layer Hot-Swap Lifecycle (Approval-Gated)



## 9.10 Action and Solution Evaluation

Each decision is linked to action outcomes by correlation id and request id. Effectiveness is scored by:

$$U = w_q \Delta q - w_t \Delta t - w_c \Delta c + w_r R$$

where:

- $\Delta q$  is quality improvement,
- $\Delta t$  is time increase,
- $\Delta c$  is cost increase,
- $R$  is resolution success indicator,
- $w_q, w_t, w_c, w_r$  are policy weights.

Outcomes are classified as effective, neutral, or regressive.

### 9.11 Storage, Indexing, and Retention

- Append-only telemetry store (SQLite first, JSON payload field).
- Core indexes: event\_type, tick, machine\_id+tick, job\_id, request\_id, timestamp\_utc.
- Retention tiers: short-term high detail, long-term aggregated archives.
- Redaction policy for sensitive operator text and model payloads.

### 9.12 Phased Rollout

**Phase A:** approval request model and token enforcement for all hot-swap endpoints.

**Phase B:** mandatory hot-swap lifecycle and approval audit events.

**Phase C:** persistent telemetry writer and indexed query APIs.

**Phase D:** decision-outcome evaluator and incident replay timeline.

**Phase E:** supervisor dashboards for pending approvals and outcome quality.

### 9.13 Verification Matrix

- Unit: token validation, expiry, scope match, event schema integrity.
- Integration: prove swap blocked without approval.
- Integration: approve, reject, timeout branches.
- Replay: reconstruct request -> approval -> execution -> outcome chain.
- Documentation: TeX compile and TikZ rendering checks.

## 10 Conclusion

This unified plan now combines geometry generation, toolpath planning, machining parameters, factory scheduling, telemetry-backed decision support, and strict human-governed hot-swap control. The architecture is designed for safe autonomy with explicit approval, full auditability, and measurable solution quality over time.